Questions to address in each project and deliverables.

Project 1: Quick Bed Sort (4 marks)

You are given a set of runs (in the form of bed files) corresponding to a sample. For example, `shuf.a.bed.gz` and `shuf.b.bed.gz` correspond to a sample named `X`. You job is to write a set of scripts (it can be a combination of bash scripts and python codes) to output a file in directory `sorted_bed_file_per_sample` that will contain all the lines from shuf.a.bed.gz and shuf.b.bed.gz that pass the chromosome selection criteria (for instance, standard_selection.tsv containing chr1 to chr22, chrX and chrY – one chromosome name in each line).

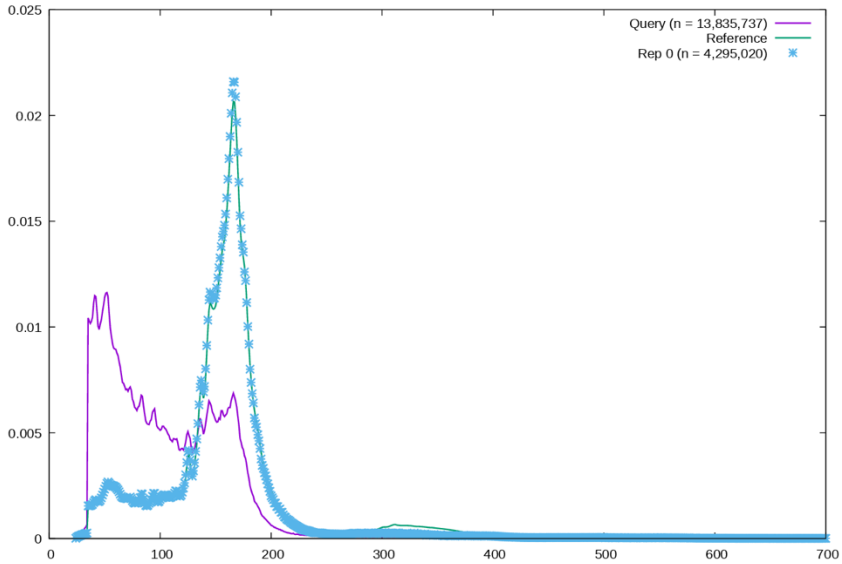
The output bed file, **however**, must be sorted in the order as you have in the selection file, meaning that if `chr10` appears after `chr1` then in the sorted sample bed file, it should also be in that order. So, apart from the bed files, you also have a selection file as a variable. Think about the name of the final output file that incorporates sample (i.e. X) and selection (i.e., standard).

The deliverable is a public GitHub repository with documentation explaining how to run the scripts.

Links for the files: https://figshare.com/s/727f8d920a1b8415f09a

Project 2: Rescale to Reference (5 marks)

You are given the same bed files as Project 1 and reference distribution file (see the green line in the plot below; the X-axis is the fragment length, and the Y-axis is the normalized frequency). Your goal is to rescale the purple line (the fragment length distribution of the query) by sampling lines from the query bed file in such a way that the resultant distribution post-sampling matches the reference. Ideally, it should match, but it is possible that some points do miss out (as you can see between 300 and 400).



The deliverable in this project is a public GitHub repo with documentation on how to run the pipeline.

Links for the files: https://figshare.com/s/727f8d920a1b8415f09a (see reference.hist and query.bed.gz)

Project 3: V-plot (3 marks)

This project is the simplest of all. Just write a Python code to take the data as standard input and generate a matrix file to plot using Gnuplot. The matrix file can be in two forms:

Wide form:
1st column – cfDNA fragment length
2nd to 1000th column – number of times a fragment length appeared at offset, for example, -500 (2nd column), -499 (3rd column) …

Long form (x, y, z):
-500 65 10
-499 65 7
…

For example, 10 is the count for fragment length 65 at -500.

Once you have this file (either the wide or the long form), you can use Gnuplot to plot this as a matrix.

Link for the data files: https://doi.org/10.6084/m9.figshare.27073276

Project 4: The PDF has already been shared, and it will be discussed in class. (**4 marks**)